

第九模块：系统架构设计与工程表达能力（嵌入式装备方向） — 超详细学习内容

本模块目标： 将前面所有技术模块整合为完整系统能力，从“会写模块代码”提升到“能够设计整体架构并清晰表达技术方案”。重点包括系统分层、任务划分、数据流设计以及工程化表达方式。

A. 系统架构基础（Architecture Thinking）

A 1 . 什么是系统架构

系统架构不是代码目录，而是：

- 模块职责划分
- 数据流方向
- 控制流逻辑
- 异常处理路径

优秀架构特点：

- 可扩展
 - 可维护
 - 可测试
-

A 2 . 常见嵌入式分层模型

推荐结构：

- Driver Layer（驱动层）
- Middleware Layer（中间件）
- Service Layer（服务层）
- Application Layer（应用层）

核心原则：

上层不直接访问寄存器。

A 3 . 模块解耦设计

方法：

- 接口抽象
- 回调机制
- 消息队列

目的：

减少模块之间的直接依赖。

B. 数据流与控制流设计

B 1 . 数据流设计思路

一个稳定系统必须明确：

数据从哪里来 → 如何处理 → 去哪里。

推荐方式：

- 画数据流图（Data Flow Diagram）。
-

B 2 . 控制流设计

常见模式：

- 状态机控制
- 事件驱动
- 周期任务调度

重点：

避免多个模块同时控制同一资源。

B 3 . 任务划分原则

建议按照功能拆分：

- 采集任务
- 控制任务
- 通信任务

- 日志任务

优先级原则：

控制 > 采集 > 通信 > 日志。

C. 进阶架构设计（工程级思维）

C 1 . 可扩展性设计

方法：

- 使用接口结构体
- 函数指针抽象硬件

例如：

不同传感器使用统一接口读取数据。

C 2 . 错误处理架构

推荐设计：

- 统一错误码
 - 统一日志入口
 - 状态机恢复机制。
-

C 3 . 低耦合高内聚原则

目标：

- 模块内部功能集中
 - 模块之间依赖最少。
-

C 4 . 系统时序设计

高级工程师思维：

不仅要看功能，还要看时间关系。

建议：

绘制时序图分析执行顺序。

D. 工程表达能力（技术沟通核心）

D 1 . 如何描述一个系统

推荐顺序：

1) 整体目标 2) 系统分层 3) 数据流 4) 关键技术点 5) 异常处理方式

D 2 . 技术文档结构建议

一个完整技术说明通常包含：

- 系统概述
 - 架构图
 - 模块说明
 - 时序说明
 - 异常处理策略
-

D 3 . 架构图绘制原则

注意：

- 不要画代码流程图
 - 重点展示模块关系与数据方向。
-

D 4 . 常见表达误区

避免：

- 只描述函数名
- 只说实现细节

应强调：

- 设计思路与权衡。
-

E. 高频技术考点（标准答案）

E 1 . 为什么系统要分层？

标准答案：

降低耦合，提高可维护性与可移植性。

E 2 . 如何保证系统可扩展？

标准答案：

通过接口抽象与模块解耦，使新增功能不影响已有模块。

E 3 . 为什么控制任务优先级最高？

标准答案：

控制系统对实时性要求最高，必须优先获得CPU资源。

E 4 . 如何描述复杂系统设计？

标准答案：

从整体架构开始，再讲模块划分与数据流，最后说明关键设计选择。

F. 注意事项（工程经验）

- 1) 不要把系统设计成单一巨大任务。
 - 2) 避免模块之间直接互相调用过多。
 - 3) 接口要稳定，避免频繁修改。
 - 4) 架构图优先于代码说明。
 - 5) 表达时先讲“为什么”，再讲“怎么做”。
-

G. 深度练习建议

- 设计一个完整嵌入式系统架构图。
- 为现有项目写一份分层说明文档。
- 画出控制任务与通信任务的时序图。

完成本模块后，你将具备系统级嵌入式架构设计与工程表达能力，实现从模块开发到整体系统设计的升级。